

Single Layer Perceptron

Lab (8)

- **1. Iris Flower Classification (Multi-class)**
 - Dataset: `sklearn.datasets.load_iris()`
 - Objective: Classify iris species (Setosa, Versicolor, Virginica).
 - Tasks:
 1. Load data, split into train/test (80/20).
 2. Standardize features.
 3. Build a Perceptron with 3 output neurons (softmax activation).
 4. Use `sparse_categorical_crossentropy` loss.
 5. Train for 200 epochs, batch size=5.
 6. Report test accuracy and confusion matrix.

- **2. Breast Cancer Diagnosis (Binary)**
 - **Dataset:** `sklearn.datasets.load_breast_cancer()`
 - **Objective:** Predict malignant (1) vs. benign (0) tumors.
 - **Tasks:**
 1. Split data (75/25), normalize features.
 2. Build a Perceptron (sigmoid output).
 3. Use `binary_crossentropy` loss.
 4. Train for 150 epochs, batch size=16.
 5. Evaluate precision, recall, and ROC-AUC.

- **3. Pima Indians Diabetes (Binary)**
 - **Dataset:** [UCI Pima Diabetes](#)
 - **Objective:** Predict diabetes onset (1: yes, 0: no).
 - **Tasks:**
 1. Handle missing values (replace 0s with mean).
 2. Split data (70/30), scale features.
 3. Build a Perceptron (sigmoid output).
 4. Use Adam optimizer (lr=0.01).
 5. Train for 100 epochs, monitor validation loss.

- **4. Wine Recognition (Multi-class)**

- Dataset: `sklearn.datasets.load_wine()`
- Objective: Classify wine cultivars (3 classes).
- Tasks:
 1. Split data (80/20), standardize.
 2. Build a Perceptron with softmax output (3 units).
 3. Use `categorical_crossentropy` loss (one-hot encode labels).
 4. Train for 250 epochs, batch size=8.
 5. Visualize loss curve and test accuracy.

- **5. Boston Housing Prices (Regression)**

- **Dataset:** `sklearn.datasets.fetch_california_housing()` (*Note: Original Boston deprecated*)
- **Objective:** Predict median house value.
- **Tasks:**
 1. Split data (75/25), scale features (StandardScaler).
 2. Build a Perceptron with linear output (1 neuron).
 3. Use `mean_squared_error` loss.
 4. Train for 100 epochs, batch size=32.
 5. Calculate test RMSE and R^2 score.

- **6. MNIST Digit Recognition (Multi-class)**

- Dataset: `keras.datasets.mnist.load_data()`
- Objective: Classify handwritten digits (0-9).
- Tasks:
 1. Flatten 28x28 images to 784-pixel vectors.
 2. Normalize pixel values [0, 1].
 3. Build a Perceptron with 10 softmax outputs.
 4. Use `categorical_crossentropy` loss.
 5. Train for 50 epochs, batch size=128.
 6. Report test accuracy and visualize misclassifications.

- **7. Titanic Survival (Binary)**

- Dataset: [Titanic CSV](#)
- Objective: Predict passenger survival (1: survived, 0: died).
- Tasks:
 1. Preprocess: Handle missing values, encode gender (male=0, female=1), drop non-numeric features.
 2. Split data (70/30), scale.
 3. Build a Perceptron (sigmoid output).
 4. Use SGD optimizer (lr=0.001).
 5. Train for 200 epochs, plot accuracy progression.

- **8. Spam Email Detection (Binary)**

- Dataset: [UCI Spambase](#)
- Objective: Classify emails as spam (1) or not (0).
- Tasks:
 1. Load data, split (80/20), scale features.
 2. Build a Perceptron (sigmoid output).
 3. Use binary_crossentropy loss.
 4. Train for 50 epochs, batch size=64.
 5. Report F1-score and precision-recall tradeoff.

- **9. Banknote Authenticity (Binary)**

- Dataset: [UCI Banknote Authentication](#)
- Objective: Detect forged banknotes (1) vs. authentic (0).
- Tasks:
 1. Split data (75/25), standardize.
 2. Build a Perceptron (sigmoid output).
 3. Use Adam optimizer (lr=0.005).
 4. Train for 100 epochs, use 20% validation split.
 5. Visualize decision boundary via PCA.

- **10. Diabetes Progression (Regression)**

- Dataset: `sklearn.datasets.load_diabetes()`

- Objective: Predict disease progression.
- Tasks:
 1. Split data (70/30), scale features.
 2. Build a Perceptron (linear output, 1 neuron).
 3. Use mean_absolute_error loss.
 4. Train for 200 epochs, batch size=10.
 5. Plot predictions vs. actual values.